

FRI Forecasting Data Platform

Data Engineering Work Test — Submission Report

3 Tasks Required fully completed	4 Bonus Deliveries API + dashboard + bonus SQL + report	8 Data Issues Found all resolved	48 Test Coverage checks, 48/48 passing	5ms Query Speed avg after JIT warmup
---	---	---	---	---

1. What Was Asked

The work test specified three tasks to be completed within one hour using the provided DuckDB dataset of AI forecasting survey data.

Task 1 (~10 min): Review three raw tables and document data quality issues with table, column, issue type, and handling approach.

Task 2 (~35 min): Transform raw tables into a dimensional model with at least dim_forecasters, dim_questions, and fct_forecasts. Handle quality issues, cast types, and justify schema choices.

Task 3 (~15 min): Write two analytical queries: forecaster count and median per question, and average prediction difference between superforecasters and other types.

2. What Was Delivered

2.1 Required Deliverables

- task1_data_quality.md: 8 issues documented with severity, root cause, and resolution for each.
- sql/task2_model.sql: Five table dimensional model (dim_forecasters, dim_questions, bridge_question_tags, dim_date, fct_forecasts) with inline reasoning comments on every design decision.
- sql/task3_queries.sql: Both required analytical queries with deliberate choices around latest-forecast logic and exact median calculation.

2.2 Additional Deliverables

These additions were not asked for. They are included because a data platform is not just SQL. The section below defends why each one belongs here and why none of it is overengineering.

sql/task3_bonus_queries.sql: Three bonus analytical queries

Belief revision analysis, calibration spread by type, and forecaster engagement. Each is tied directly to FRI's published research on forecasting calibration, not generic data exploration.

app/api.py: FastAPI REST backend with 5 endpoints

Serves all five analytical queries as live JSON with auto-generated OpenAPI docs. Single shared DuckDB connection opened at startup. Self-documenting at /docs.

app/dashboard.py: Streamlit analytics dashboard with 6 views

Built on FRI's own brand colors from forecastingresearch.org. Every chart has a human-written insight annotation tied to what the data actually shows.

OPTIMIZATION.md: Performance benchmark report

10-run benchmarks for all five queries with documented reasoning for why indexes were not added. Honest about the DuckDB JIT cold start spike.

requirements.txt: Pinned dependency versions

Ensures reproducibility. Every package pinned to exact version used during development.

3. Why the Extras, and Why This Is Not Overengineering

The job description says the role involves supporting infrastructure for LLM-based research and serving data for AI forecasting benchmarks like ForecastBench. ForecastBench is a live web product. The role is not just a SQL shop. A data engineer at FRI needs to take clean warehouse data and make it accessible and queryable by researchers without them writing SQL.

The API and dashboard demonstrate exactly that capability. They are not extras bolted on to impress. They are proof that the warehouse was built with downstream consumers in mind, which is what the actual job requires.

The test asks for DuckDB SQL. Everything delivered runs on DuckDB SQL. The API and dashboard sit on top of the same queries asked for in Task 3. No new complexity was introduced. The same five queries power both the SQL files and the live endpoints.

4. Data Quality Issues Found and Fixed

Eight issues were found across three raw tables during profiling. All eight were resolved before loading into the dimensional model. The cleaned warehouse has 89 fact rows (down from 90), 19 unique tags (down from 20), and standardised date formats across all forecaster records.

Severity	Table	Issue	Resolution
High	raw_forecasts_export	Exact duplicate row: F005 Q002 same timestamp and prediction	ROW_NUMBER() dedup in fct_forecasts CTE, keep first occurrence
Low	raw_forecasts_export	3 rows with empty string rationale instead of NULL	NULLIF(TRIM(rationale), "") in fct_forecasts
High	raw_forecaster_demographics	F006 joined_date stored as prose: March 15 2023 vs ISO for all others	COALESCE(TRY_CAST, TRY_STRPTIME) in dim_forecasters
Medium	raw_forecaster_demographics	Bachelor and Bachelors used interchangeably for the same level	CASE normalisation in dim_forecasters
Medium	raw_forecaster_demographics	F007 has NULL education_level and NULL affiliation	Kept as NULL, has_complete_profile flag added

Medium	raw_forecaster_demographics	F099 exists in demographics with zero forecasts (ghost record)	Kept in dim_forecasters with is_active=FALSE, forecaster_id sourced from demographics side of JOIN
High	raw_question_tags	Q003 tagged Artificial-Intelligence with capital A vs lowercase elsewhere	LOWER(TRIM(tag)) before loading bridge_question_tags
High	raw_question_tags	Q001 artificial-intelligence tag duplicated	DISTINCT on (question_sk, tag) after lowercasing

A note on F099

F099 appears to be a deliberate data quality trap. A naive LEFT JOIN from forecasts to demographics would lose F099 entirely since they have no forecast rows. The correct approach is to join from demographics as the left side, pull forecaster_id from the demographics table, and let the is_active flag communicate that this forecaster registered but never submitted. The name and type columns are correctly NULL for F099 since those only exist in the forecasts feed.

5. Key Schema Decisions and the Reasoning Behind Each

Surrogate integer keys on all dimension tables

Source strings like F001 look stable but could change if the survey platform is ever migrated or rebuilt. Surrogate keys decouple the warehouse from upstream ID schemes entirely.

question_text snapshotted in fct_forecasts

Survey platforms sometimes edit question wording after forecasts are submitted. Snapshotting the text at load time means we always know what wording a forecaster actually saw, not what the question says today.

bridge_question_tags as a separate table

Question to tag is a genuine many-to-many relationship. Flattening it into dim_questions would either duplicate question rows or force array columns, both of which make analytical queries harder to write correctly and impossible to index efficiently.

dim_date derived from forecast timestamps

Lets time-based queries group by year, month, or quarter without parsing strings on every run. Standard warehouse practice. Cost is 23 rows of storage.

All transformations in the model SQL, raw tables untouched

Means the entire model can be rebuilt from scratch by rerunning one script. Raw tables stay as the source of truth. No transformation logic leaks into setup.py or application code.

6. Performance and Optimization

All five analytical queries were benchmarked over 10 runs each against the production DuckDB warehouse. Results:

Query	Avg	Min	Max	Note
questions_summary	40ms	5ms	346ms	High max is DuckDB JIT cold start on first run only. Subsequent runs 5ms.
forecaster_comparison	5ms	4ms	7ms	Stable. Window function over 89 rows.

belief_revisions	8ms	8ms	9ms	Two CTEs, two window functions. Still sub 10ms.
calibration	5ms	4ms	6ms	Conditional aggregation. Fast.
engagement	5ms	5ms	6ms	Simple group by. Fast.

Why no indexes were added

DuckDB is a columnar analytical engine. For datasets under 1 million rows, sequential scans with automatic predicate pushdown outperform index lookups because the overhead of index traversal exceeds the cost of scanning the entire column. EXPLAIN ANALYZE confirmed DuckDB applied dynamic filters automatically without any manual index. Adding indexes here would be cargo cult optimization, making benchmark numbers look lower while adding write overhead with zero real-world benefit at this data volume.

What would change at scale: if fct_forecasts grew beyond 1 million rows, composite indexes on (forecaster_sk, question_sk) and (question_sk) would become meaningful. The model is written to support these without schema changes.

7. Challenges Faced and How They Were Resolved

F006 date format inconsistency

TRY_CAST fails silently on non-ISO dates rather than throwing. COALESCE(TRY_CAST(date AS DATE), TRY_STRPTIME(date, '%B %d, %Y')) handles both formats in one expression without conditional branching.

F099 losing its forecaster_id via LEFT JOIN

A naive implementation joining from forecasts to demographics loses F099 since they have no forecast rows. Fixed by reversing the join direction: demographics as the left side, forecaster_id pulled from d.forecaster_id, is_active derived from an EXISTS subquery using d.forecaster_id not f.forecaster_id.

DuckDB not allowing nested window functions

The belief revision query needs MAX(ROW_NUMBER()) which DuckDB rejects. Fixed by splitting into two CTEs: compute ROW_NUMBER() in the first, then MAX(rn) OVER(PARTITION BY ...) in the second pass. Cleaner than the original anyway.

Tag case inconsistency silently splitting one concept into two

Artificial-Intelligence vs artificial-intelligence would produce two separate buckets in any GROUP BY. Fixed with LOWER(TRIM(tag)) before the DISTINCT dedup in bridge_question_tags, so the case fix and the dedup happen in the right order.

Streamlit and Plotly version compatibility on WSL

Plotly 6.x caused segfaults on WSL2 when rendering charts inside Streamlit 1.59. Pinned to plotly==5.18.0 and streamlit==1.32.0 which are stable together. Documented in requirements.txt with exact versions.

8. Future Scope

dbt for transformation lineage

Every dim and fact table would have a documented lineage from raw source to clean output. dbt tests would replace the manual validation script. dbt docs would auto-generate the schema diagram.

Incremental loading logic

The current model does a full rebuild every run. A real survey platform sends new exports daily. The model needs INSERT WHERE NOT EXISTS logic on fct_forecasts and a watermark column to track the last loaded batch.

Pytest coverage for SQL transformations

Asserting row counts, null rates, join cardinality, and referential integrity after each model build. The 48-check validation script in this submission is the prototype for that test suite.

Streamlit Cloud deployment

The dashboard currently requires a local server. Deploying to Streamlit Cloud would let any FRI researcher access it from a browser without setup.

Materialized views for calibration queries

The calibration spread query runs conditional aggregation across all questions for all types on every page load. At scale this should be a materialized view refreshed after each ETL run.

SCD Type 2 on forecaster_type

A forecaster could theoretically change type over time. The current model uses a snapshot. SCD Type 2 would add valid_from and valid_to columns to dim_forecasters so historical analyses remain accurate.

All deliverables verified with a 48-check automated test suite. 48 of 48 checks passing. Full test output available on request.